LINUX PRIVILEGE ESCALATION AUTOMATION TOOLKIT

A Defensive Linux Misconfiguration Audit & Reporting Project

CYBERSECURITY PROJECT REPORT

Prepared by: UMMEDA VIVEK

Domain: Cyber Security • Linux Security • Red Team Foundations

Project type: Local, read-only security enumeration and risk reporting

Submission template — add your genuine screenshots and replace all sample/blank results with observations from your own authorized lab.

Project Approval & Declaration

Certificate (for institute / mentor use)

This is to certify that the project titled **“Linux Privilege Escalation Automation Toolkit”** was prepared by **UMMEDA VIVEK** as an educational activity in Cyber Security / Ethical Hacking. The project reviews selected Linux configuration indicators and produces a local audit report. It does not automatically exploit findings or change system settings.

Mentor / Guide: _____

Institute: _____Unified Mentor_____

Date: _____30/10/2026_____ Signature: ____u.vivek_____

Student declaration

I declare that I will run the toolkit only on systems I own or have explicit permission to assess. I will verify every result, protect sensitive output, and distinguish confirmed observations from potential risks. I understand that a finding is not proof that privilege escalation is possible.

Student: UMMEDA VIVEK Signature: ____u.vivek_____ Date: __30/10/2026___

Submission note

The certificate area is a template only. It must be completed and signed by an authorized institute representative if required. Do not present it as an already-awarded certificate.

Contents

Section	Title
01	Abstract and Executive Summary
02	Introduction and Background
03	Problem Statement
04	Aim, Objectives and Scope
05	Requirements and Tools
06	System Architecture
07	Workflow and Module Design
08	Security Checks Covered
09	Risk Scoring and Report Format
10	Implementation Plan and Core Logic
11	Setup and Execution Guide
12	Test Plan
13	Results and Analysis
14	Screenshots and Evidence — 14 Required Images
15	Security, Ethics and Limitations
16	Future Enhancements
17	Conclusion
18	References
19	Final Submission Checklist

How to use this report

Read the report, build/run the toolkit in an authorized Linux lab, capture the 14 genuine screenshots described in Section 14, and replace blank result fields with actual findings. Never claim that an untested check passed.

01 | Abstract and Executive Summary

The Linux Privilege Escalation Automation Toolkit is a beginner-friendly defensive auditing project. It collects selected local Linux configuration indicators that may deserve review, such as SUID/SGID files, writable paths, scheduled-task permissions, sudo policy information, and basic system details. It organizes the observations into a human-readable report so an administrator or security learner can prioritize manual verification.

The toolkit is intentionally designed as a **read-only checker**: it does not attempt to gain root access, run exploit code, alter permissions, create persistence, or modify system configuration. It is a triage aid, not a replacement for a full security assessment.

Expected deliverables

• A working local Python-based audit script (or equivalent shell/Python implementation).

• A structured findings report with check name, evidence, risk level, and recommended remediation.

• A test log showing which checks ran and any errors or skipped checks.

• Fourteen real screenshots inserted in the evidence section.

• Source code, README, and this final report stored in a GitHub repository if required.

02 | Introduction and Background

Linux systems use accounts, groups, file permissions, scheduled tasks, service configurations, and administrative policies to control access. Privilege escalation risk can arise when a configuration allows an unprivileged user to influence a process or file that is later used by a more privileged account. The existence of a configuration indicator alone does not prove that escalation is possible; context and manual validation are necessary.

A privilege-escalation audit commonly begins with safe enumeration: identifying the OS and kernel version, understanding current identity and group membership, reviewing administrative policy, and checking permissions on selected files and directories. This project automates a small, transparent subset of those observations.

Key terms

Term	Meaning in this project
Privilege escalation	A change from a lower-privileged context to a more privileged one.
SUID / SGID	Permission bits that can affect the effective identity used when a program runs; each result needs contextual review.
Writable path	A file or directory that the current user can modify; risk depends on ownership and how privileged processes use it.
Cron / scheduled task	A job configured to run at a particular time or interval; ownership and write permissions matter.
Least privilege	Granting only the access needed to perform a task.
False positive	A flagged item that is not actually exploitable or relevant after review.

03 | Problem Statement

Manual Linux enumeration can be repetitive and inconsistent for a beginner. Notes may be scattered across terminal output, and it can be difficult to explain why a configuration deserves review. A small automated tool can standardize collection, label findings, and make the evidence easier to review.

Research question: Can a read-only local toolkit collect selected Linux security configuration indicators, present them consistently, and provide practical remediation guidance without changing the system?

Proposed solution

1. Collect basic host and current-user information using standard, non-destructive commands or Python libraries.

2. Inspect a defined set of configuration indicators and record evidence rather than attempting exploitation.

3. Assign preliminary severity based on a documented rule set.

4. Generate a readable report with findings, caveats, and remediation suggestions.

5. Validate the tool using safe test fixtures and compare output against expected behavior.

Success criteria

• The script completes without modifying system configuration.

• Each check reports PASS, REVIEW, SKIPPED, or ERROR with a short reason.

• The report clearly distinguishes observed evidence from risk interpretation.

• Test cases and actual results are documented, including failures and limitations.

04 | Aim, Objectives and Scope

Aim

To build a local Linux auditing toolkit that automates selected privilege-escalation-related configuration checks and produces a useful evidence-based report for defensive review.

Objectives

• Identify the operating system, kernel, current user, and group context.

• Collect relevant permission and policy indicators without exploiting them.

• Review selected SUID/SGID entries, writable locations, scheduled-task configuration, and sudo-policy visibility where permitted.

• Classify results using transparent preliminary risk rules.

• Offer a practical remediation note for each flagged item.

• Record errors and skipped checks instead of silently ignoring them.

• Create evidence screenshots and a reproducible README.

In scope

Authorized local Linux systems and training virtual machines; read-only inventory; safe file metadata checks; reporting; test fixtures; remediation guidance.

Out of scope

Automatic exploitation, password cracking, stealth, persistence, credential collection, payload delivery, destructive changes, or testing on third-party systems without written permission. The tool does not certify that a system is secure.

05 | Requirements and Tools

Category	Requirement / recommendation
Hardware	Laptop or desktop capable of running a Linux virtual machine.
Operating system	Kali Linux, Ubuntu, Debian, or another authorized Linux lab.
Runtime	Python 3.x; standard library preferred for portability.
Editor	Any text editor or code editor.
Terminal	Linux terminal with permission to view the chosen configuration files.
Version control	Git and a GitHub repository for source and documentation, if required.
Input	Local host configuration metadata; no sensitive personal files.
Output	Terminal summary and/or text/JSON report.

Minimum safe setup

1. Create a dedicated project folder in your home directory.

2. Save the script and README in that folder.

3. Use a disposable VM or a system you own/are authorized to assess.

4. Run first without sudo; record permission-related skipped checks.

5. Only use elevated privileges when genuinely needed to read a specific file, and only with authorization.

6. Save output in the project folder and redact usernames, hostnames, IP addresses, or other sensitive details before sharing.

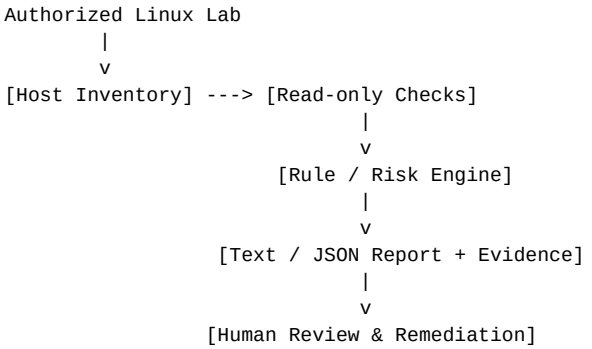
Avoid running unknown scripts as root. Read the code first and understand what each check does.

06 | System Architecture

The toolkit follows a simple pipeline: collect → inspect → classify → report. Each module has a narrow responsibility so the code is easier to explain and test.

Module	Responsibility	Example output
Host inventory	Collect OS, kernel, current user, groups where available.	Host metadata
Permission checks	Read metadata for selected files and paths.	Owner, mode, writable flag
Policy checks	Collect visible sudo-policy or scheduled-task configuration indicators when permitted.	Review item / skipped
Rule engine	Map observations to informational / low / medium / high review categories.	Preliminary risk label
Report writer	Print or save structured findings.	Text or JSON report
Error handler	Record missing files and denied access safely.	ERROR / SKIPPED + reason

Architecture diagram (text representation)



Design principle: the tool reports potential misconfigurations; a human reviewer determines whether each finding is meaningful in context.

07 | Workflow and Module Design

Workflow

1. Start the program and record a timestamp.

2. Collect basic system information and current execution identity.

3. Run each defined read-only check independently.

4. For each check, capture evidence, status, and any exception.

5. Apply the documented preliminary severity rule.

6. Generate a summary and detailed findings list.

7. Review every flagged item manually and document the actual result.

8. Save report and screenshots; redact sensitive information before publishing.

Recommended finding fields

Field	Purpose
ID	Unique identifier such as LPA-001.
Check	Short name of the check.
Status	PASS, REVIEW, SKIPPED, or ERROR.
Evidence	Minimal relevant output or metadata.
Risk	Preliminary category; not a proof of exploitability.
Why it matters	Brief explanation of the potential concern.
Recommendation	Defensive action for an administrator to validate.
Validation	Observed outcome after manual review; leave blank until checked.

08 | Security Checks Covered

Check ID	Check	What to record	Important caveat
C01	System inventory	OS release, kernel, architecture	An older version alone does not prove a vulnerability.
C02	Identity context	Current user and groups	Do not publish sensitive host details.
C03	SUID/SGID inventory	File path, owner, permission bits for a limited scan	Many entries are legitimate; review package and context.
C04	Writable sensitive paths	Ownership and write permissions on selected paths	Only risky if a privileged process relies on the path.
C05	Scheduled tasks	Ownership and write access of selected cron files/directories	Do not execute or modify jobs.
C06	Sudo policy visibility	Record whether authorized policy inspection is available	Never attempt to exploit listed permissions.
C07	PATH hygiene (basic)	Check whether PATH contains empty or relative entries	Environment and service context affect risk.
C08	Error handling	Missing files, denied reads, unsupported OS	Skipped does not mean secure.
C09	Report integrity	Ensure findings and timestamp are saved	Redact secrets before sharing.

Safe behavior requirement

Checks should inspect metadata or read known configuration files only. They must not change permissions, create files in sensitive directories, run discovered binaries, alter scheduled tasks, or attempt to obtain root privileges.

09 | Risk Scoring and Report Format

Use risk labels as a prioritization aid, not as a claim of confirmed exploitability. A beginner project should explain its rules and acknowledge uncertainty.

Label	Suggested meaning	Response
INFO	Inventory detail; no direct issue asserted.	Keep for context.
LOW	Hardening improvement or weak indicator with limited evidence.	Review during routine maintenance.
MEDIUM	Configuration deserves timely manual verification.	Confirm ownership, use, and exposure.
HIGH	Strongly concerning configuration supported by clear evidence.	Restrict access and have an administrator investigate promptly.
UNKNOWN	Insufficient permission or context.	Collect authorized evidence; do not infer PASS.

Example report format (illustrative only)

```
Linux Privilege Escalation Automation Toolkit
Run status: COMPLETE WITH REVIEW ITEMS
Check: C03 — SUID/SGID inventory
Status: REVIEW
Evidence: [insert sanitized local observation]
Risk: UNKNOWN until context is validated
Recommendation: Verify package ownership and whether the bit is required.

Note: Replace this illustrative sample with output from your own run.
```

Do not fill the final report with invented findings. If your system has no flagged items, say so and include the evidence that the check ran successfully.

10 | Implementation Plan and Core Logic

Recommended beginner implementation

Keep the code small and modular. Use Python's standard library where practical, such as os, platform, pathlib, json, datetime, and subprocess only for specific read-only commands. If a command is unavailable, record a skipped check rather than crashing.

Pseudocode

```
START
  initialize report metadata
  collect host and current-user details
  for each read-only check:
      try:
          collect minimal evidence
          apply transparent rule
          append result with status and caveat
      except permission or missing-file error:
          append SKIPPED/ERROR with reason
  save human-readable report
  optionally save JSON output
  print summary counts
END
```

Implementation checklist

• Use functions such as collect_system_info(), check_permissions(), evaluate_finding(), and write_report().

• Keep paths explicit and checks bounded; avoid scanning the entire filesystem by default.

• Quote and handle paths safely; do not construct shell commands from untrusted input.

• Use exception handling for permission-denied and missing-file cases.

• Do not use eval(), download-and-execute patterns, or automatic remediation.

• Add a version string and timestamp to each report.

• Document any command that requires additional privileges.

The final source code must match the behavior described in this report. Capture screenshots only after the script actually runs.

11 | Setup and Execution Guide

Step-by-step workflow

1. Open your authorized Linux VM and sign in.

2. Create a project directory: `mkdir -p ~/linux-privilege-audit`

3. Open that directory and create the Python script and README.

4. Review the source code line by line before running it.

5. Run the script from the project directory using Python 3.

6. Read the output and note every PASS, REVIEW, SKIPPED, and ERROR result.

7. If a check needs more access, first understand why; use only authorized read-only access.

8. Save the output to a report file if the program supports it.

9. Repeat the run and confirm the summary is consistent.

10. Take screenshots listed in Section 14 and insert them in the reserved boxes.

Example commands for creating the folder and checking Python

```
mkdir -p ~/linux-privilege-audit
cd ~/linux-privilege-audit
python3 --version
pwd
ls -la
```

Run command (replace the filename with your actual script name)

```
python3 linux_privilege_audit.py
```

If the command reports 'No such file or directory', check the current folder with pwd and list filenames with ls -la. Do not rename files randomly; use the exact filename that exists.

12 | Test Plan

Test ID	Test	Expected behavior	Actual result
T01	Script starts on supported Linux	Start banner and timestamp appear	_____
T02	Host inventory available	OS/kernel fields populated or clearly skipped	_____
T03	Permission check	Metadata recorded without changing permissions	_____
T04	SUID/SGID scan	Entries summarized; no binaries executed	_____
T05	Missing test path	SKIPPED/ERROR with reason; program continues	_____
T06	Permission denied	Access limitation reported clearly	_____
T07	Report generation	Report file created at documented path	_____
T08	JSON validity (if enabled)	Output parses as valid JSON	_____
T09	Repeat run	Stable results where system state is unchanged	_____
T10	Read-only behavior	No permissions/configuration modified	_____
T11	Sensitive output review	Host identifiers redacted before publication	_____
T12	No elevated privileges	Tool handles restricted checks safely	_____

Mark each test PASS or FAIL only after you run it. Add a short note for failures and document the fix. A test plan with blank actual-result fields is a template, not proof of successful testing.

13 | Results and Analysis

Use this section to explain what the tool actually observed on your own authorized Linux environment. Avoid claiming that a flagged item is exploitable unless you have separate authorization and sufficient evidence to validate it.

Metric	Actual value
Date/time of run	_____
Linux distribution/version	_____
Checks attempted	_____
Checks completed	_____
Checks skipped / errors	_____
INFO / LOW findings	_____
MEDIUM / HIGH items requiring review	_____
Tests passed / failed	_____

Discussion prompts

• Which checks completed successfully and which were limited by permissions?

• Were any findings expected for the distribution or installed packages?

• What evidence was used to classify each item as a review candidate?

• What remediation would you recommend, and who should validate it?

• What did the tool not check? How might those limitations affect interpretation?

14 | Screenshots and Evidence

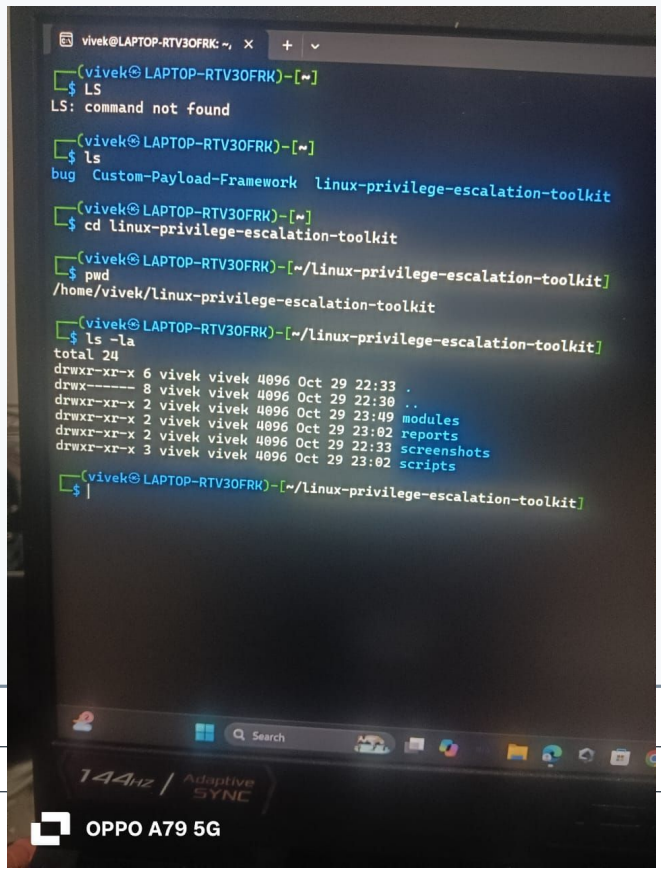
Insert 14 genuine screenshots captured from your own laptop. Each screenshot should be readable, show the relevant application/terminal, and match the description. Avoid showing passwords, tokens, personal documents, or unrelated browser tabs. The boxes on the following pages are reserved for your images.

Recommended capture method: use the system screenshot shortcut, save PNG files with the names below, and insert each image into its matching placeholder in your final editor/PDF workflow. Do not use generated or fabricated screenshots as evidence.

Screenshot 01 — Project Folder

Show the project directory with the main Python script, README, report/output folder, and evidence folder visible.

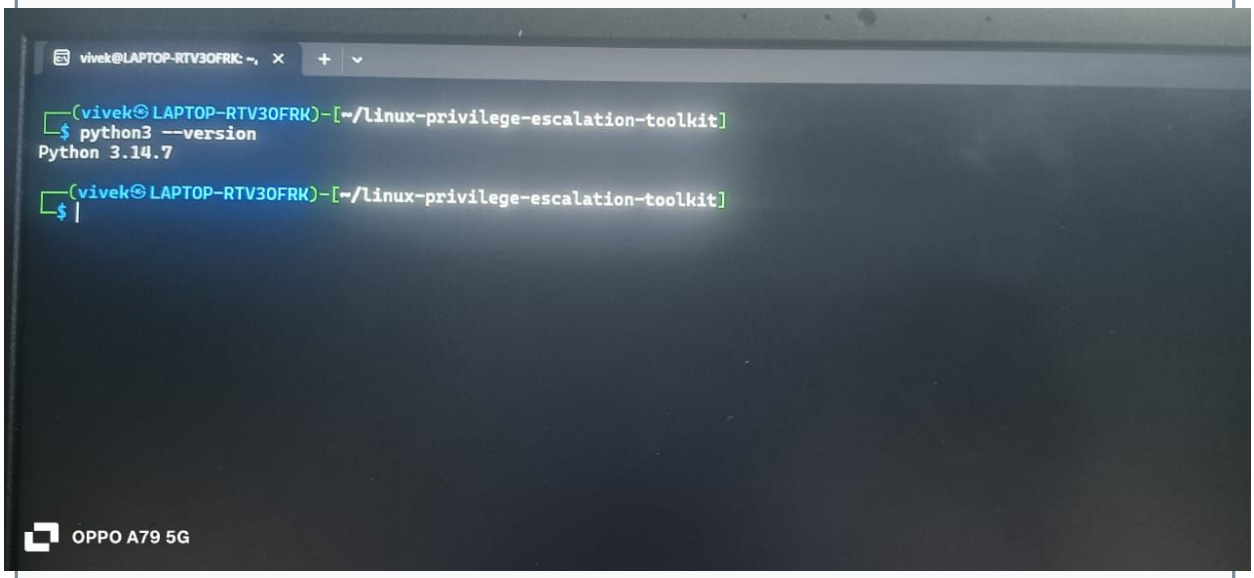
Suggested filename: 01_project_folder.png



Screenshot 02 — Python Version and Environment

Show the Linux terminal, Python 3 version, and current project directory. Hide any personal or sensitive host identifiers if needed.

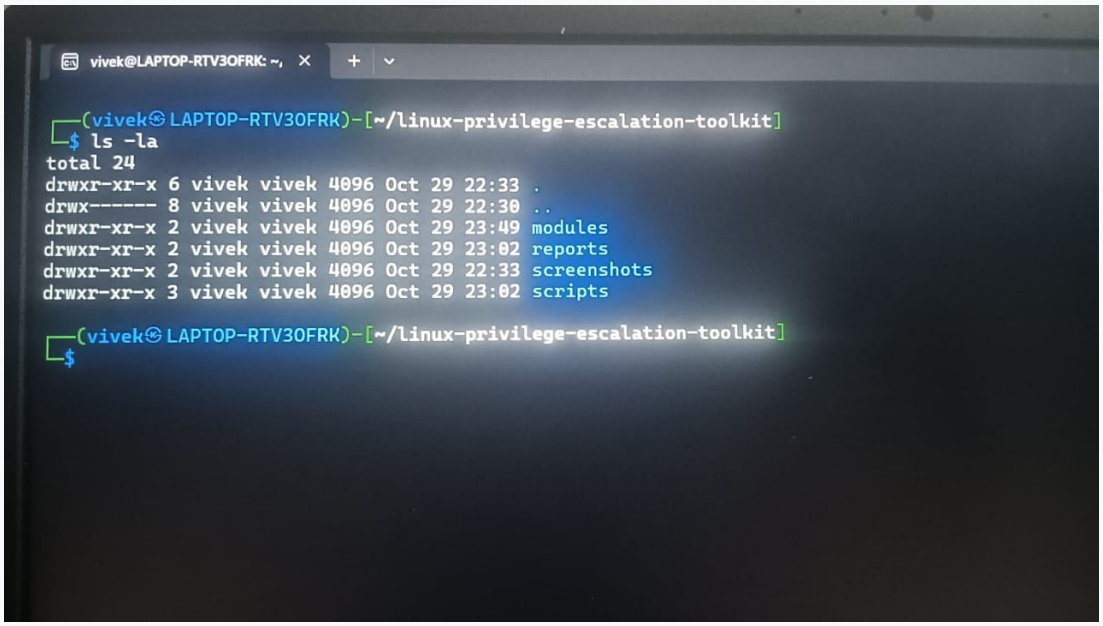
Suggested filename: 02_environment.png



Screenshot 03 — Source Code: Main Structure

Show the source code in the editor, including the main function and module/function names.

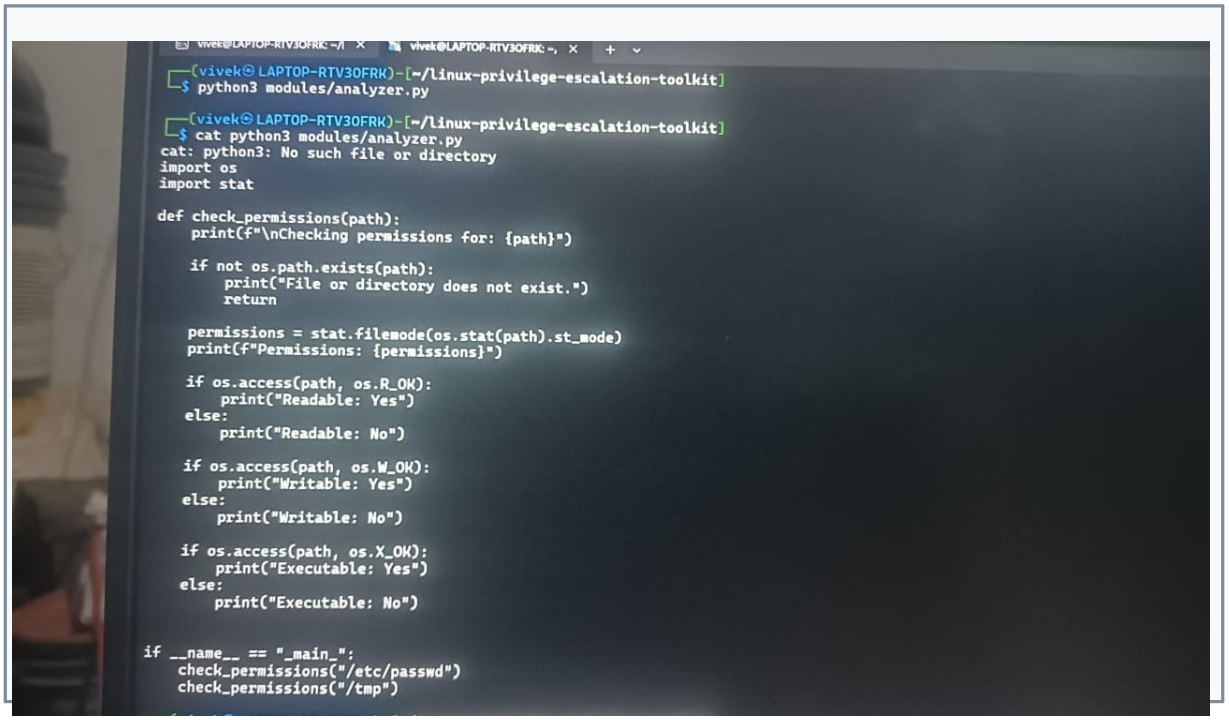
Suggested filename: 03_source_structure.png



Screenshot 04 — Source Code: Permission Checks

Show the read-only permission metadata logic. The screenshot should make clear that the code inspects rather than changes permissions.

Suggested filename: 04_permission_checks.png



```
(vivek㉿LAPTOP-RTV3OFRK)-[~/linux-privilege-escalation-toolkit]
$ python3 modules/analyzer.py

(vivek㉿LAPTOP-RTV3OFRK)-[~/linux-privilege-escalation-toolkit]
$ cat python3 modules/analyzer.py
cat: python3: No such file or directory
import os
import stat

def check_permissions(path):
    print(f"\nChecking permissions for: {path}")

    if not os.path.exists(path):
        print("File or directory does not exist.")
        return

    permissions = stat.filemode(os.stat(path).st_mode)
    print(f"Permissions: {permissions}")

    if os.access(path, os.R_OK):
        print("Readable: Yes")
    else:
        print("Readable: No")

    if os.access(path, os.W_OK):
        print("Writable: Yes")
    else:
        print("Writable: No")

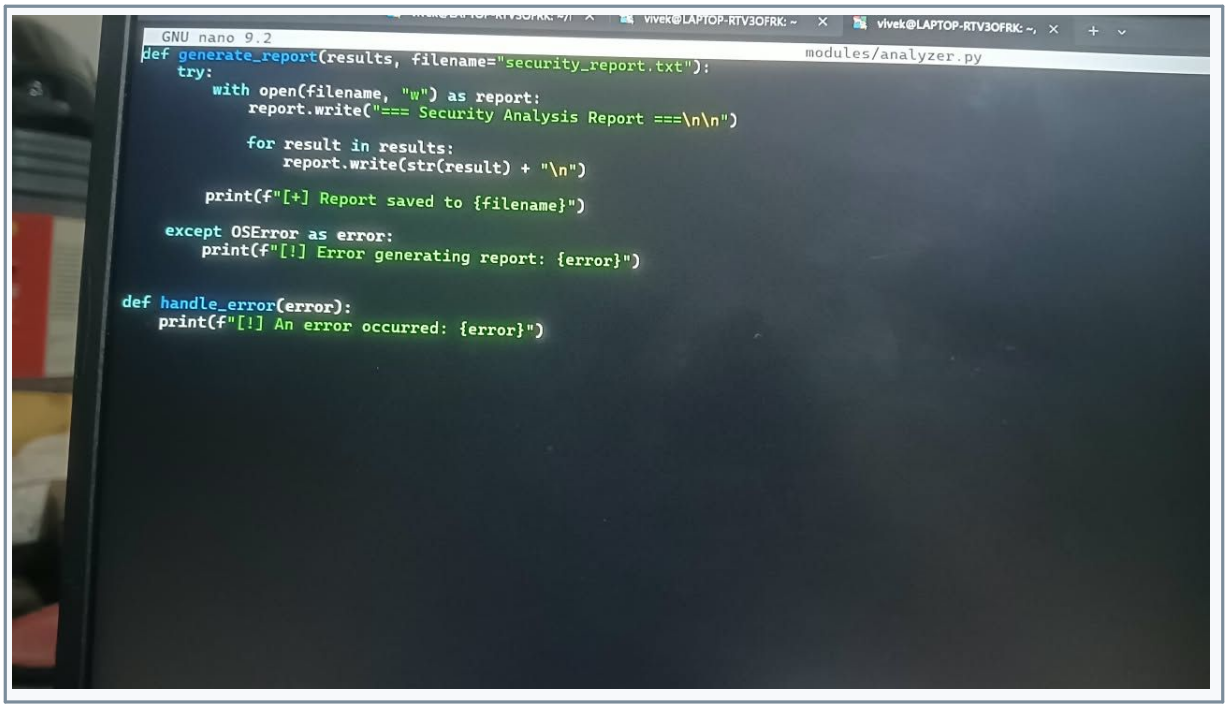
    if os.access(path, os.X_OK):
        print("Executable: Yes")
    else:
        print("Executable: No")


if __name__ == "_main_":
    check_permissions("/etc/passwd")
    check_permissions("/tmp")
```

Screenshot 05 — Source Code: Reporting and Error Handling

Show the report-writing function and handling for missing files or permission errors.

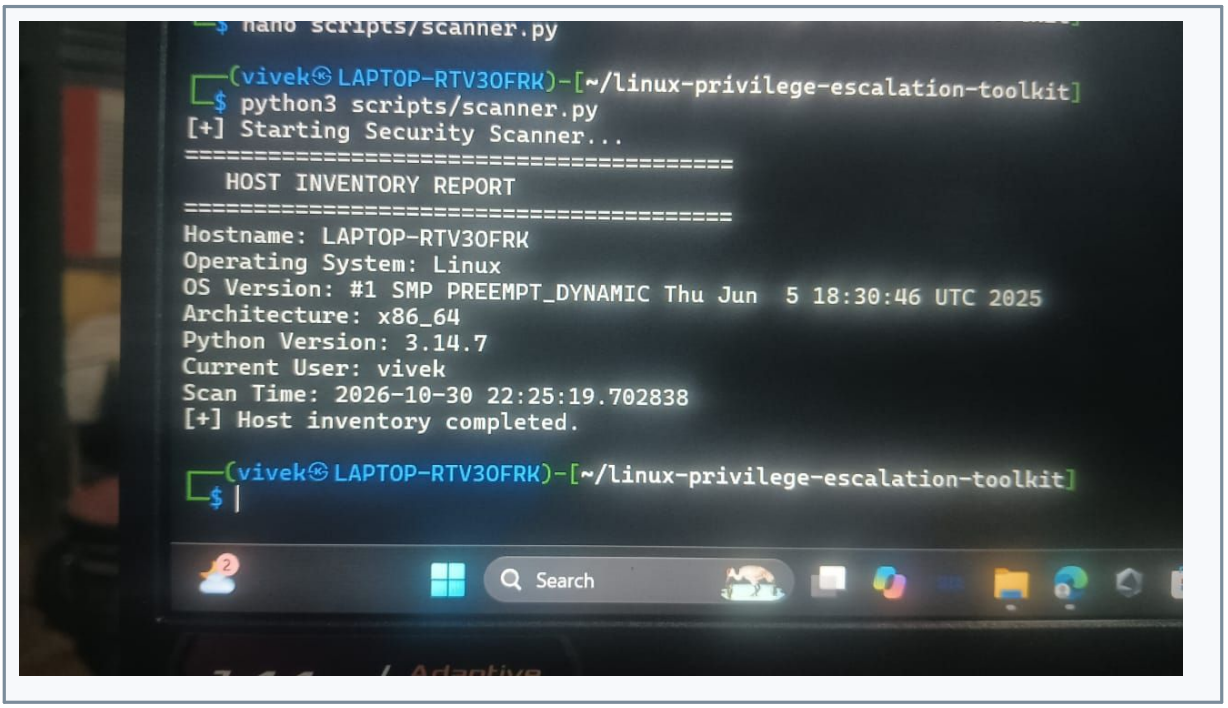
Suggested filename: 05_reporting_errors.png



Screenshot 06 — Script Start and Host Inventory

Show the exact command used to run the script and the host inventory section of actual output.

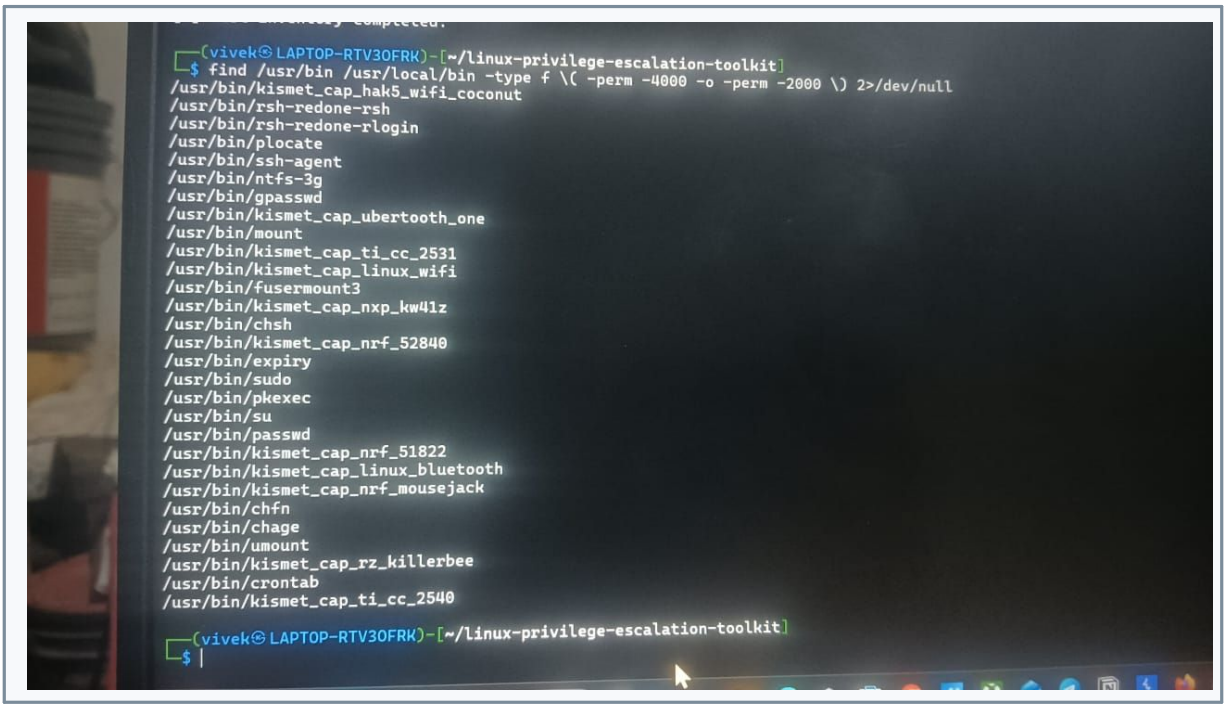
Suggested filename: 06_host_inventory.png



Screenshot 07 — SUID/SGID Check Output

Show the check summary and a small, readable section of actual results. Do not execute listed programs.

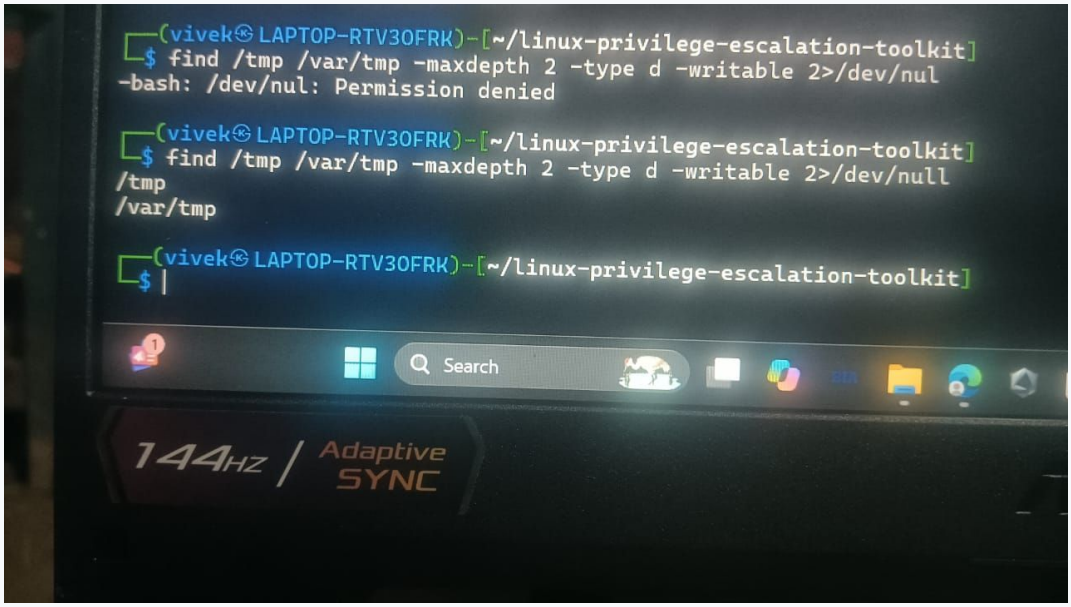
Suggested filename: 07_suid_sgid_results.png



Screenshot 08 — Writable Path / Permission Review

Show the actual permission-review output and its status. Redact sensitive paths if necessary.

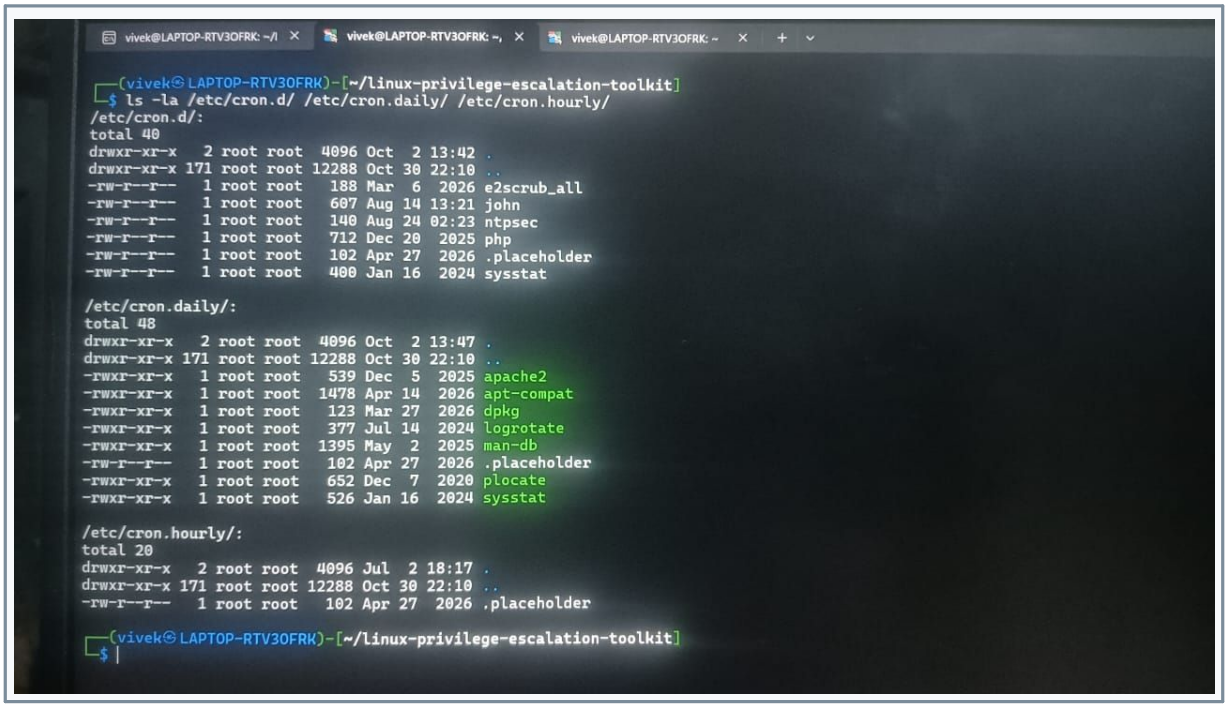
Suggested filename: 08_writable_path_review.png



Screenshot 09 — Scheduled Task Review

Show the scheduled-task check status or a clear SKIPPED message if access was unavailable. Do not edit or run tasks.

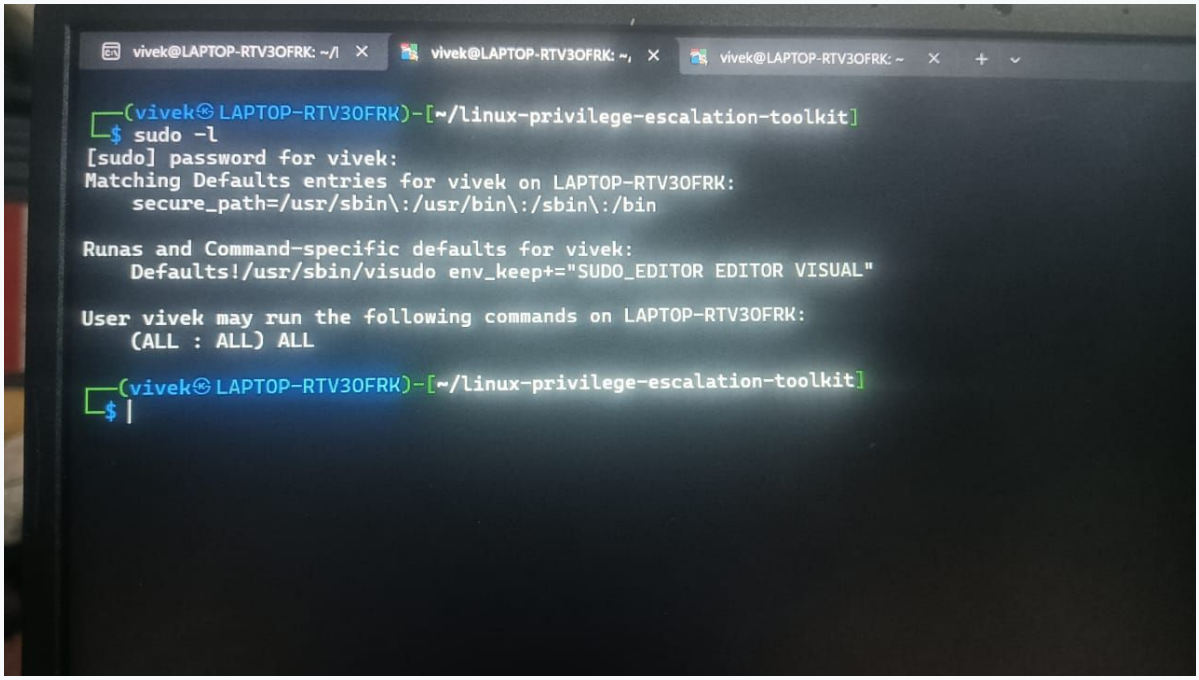
Suggested filename: 09_scheduled_tasks.png



Screenshot 10 — Sudo Policy Visibility

Show only authorized policy-inspection output or the tool's permission-limited status. Redact usernames and sensitive policy details before sharing.

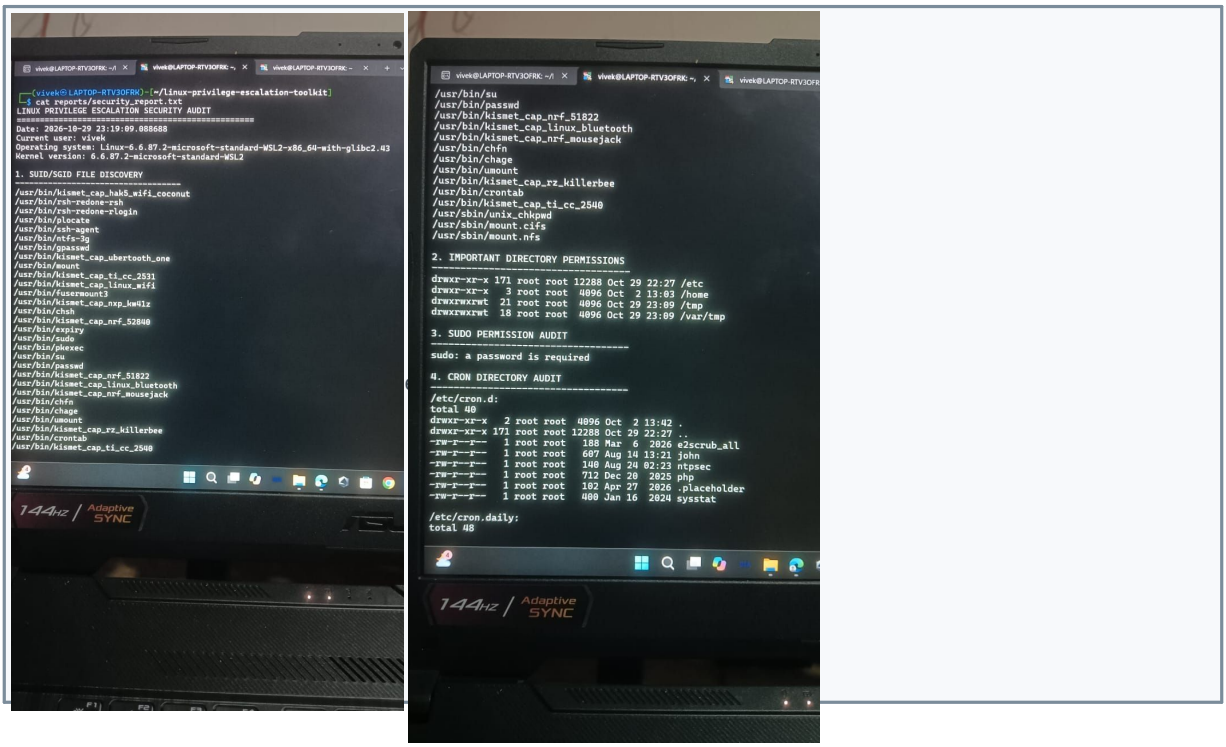
Suggested filename: 10_sudo_policy.png

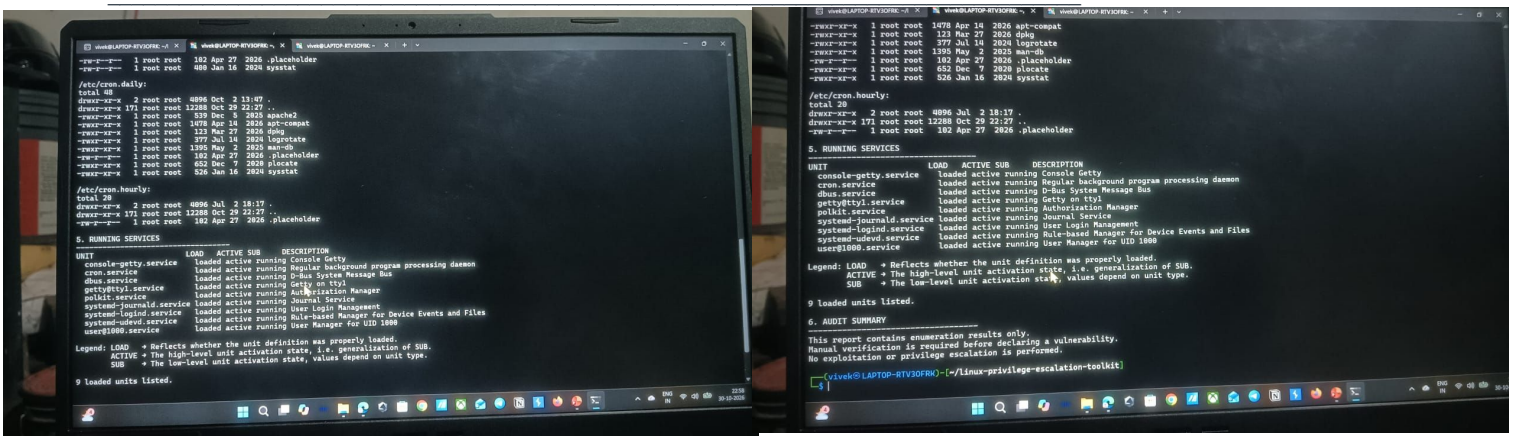


Screenshot 11 — Full Summary / Risk Labels

Show the final summary with counts and preliminary risk labels from the actual run.

Suggested filename: 11_summary.png

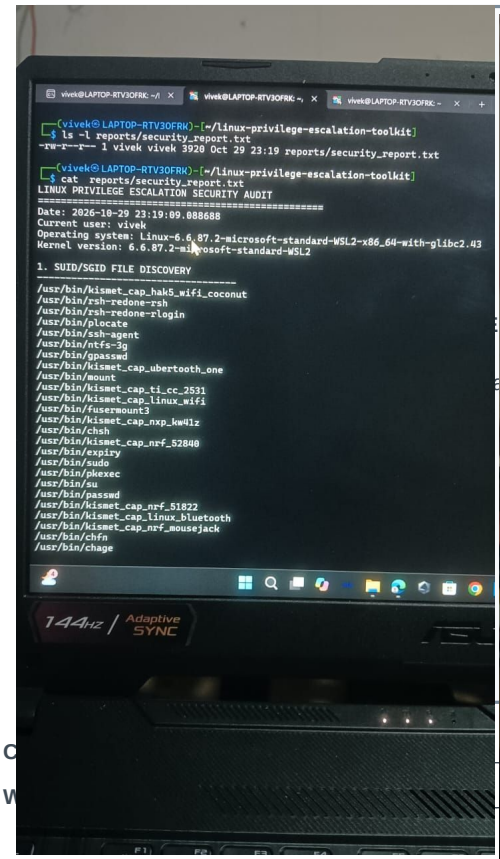


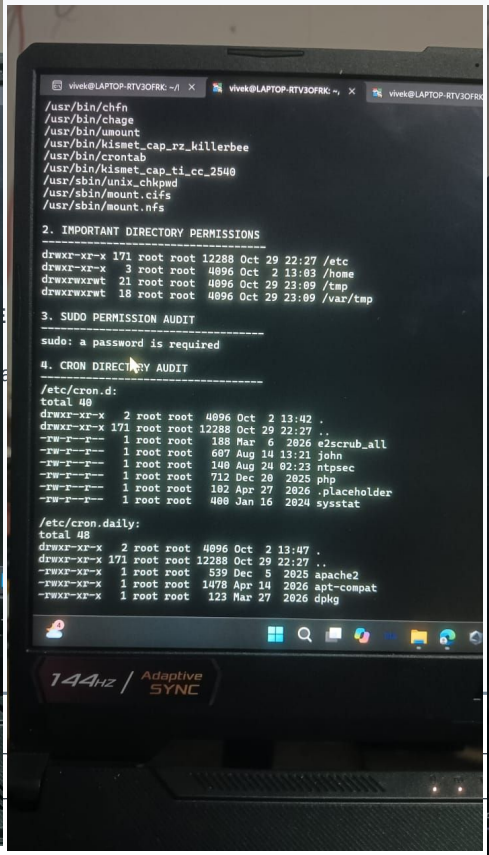


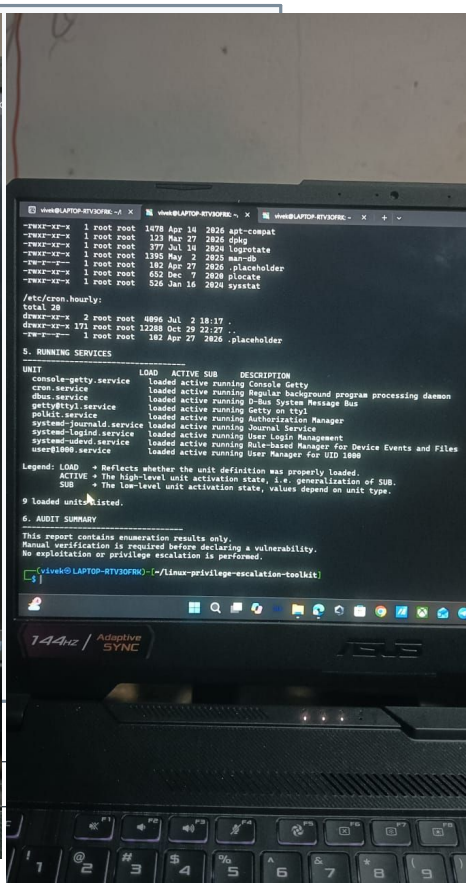
Screenshot 12 — Saved Report File

Show the generated report file in the folder and, if possible, the report opened with its timestamp and findings.

Suggested filename: 12_saved_report.png



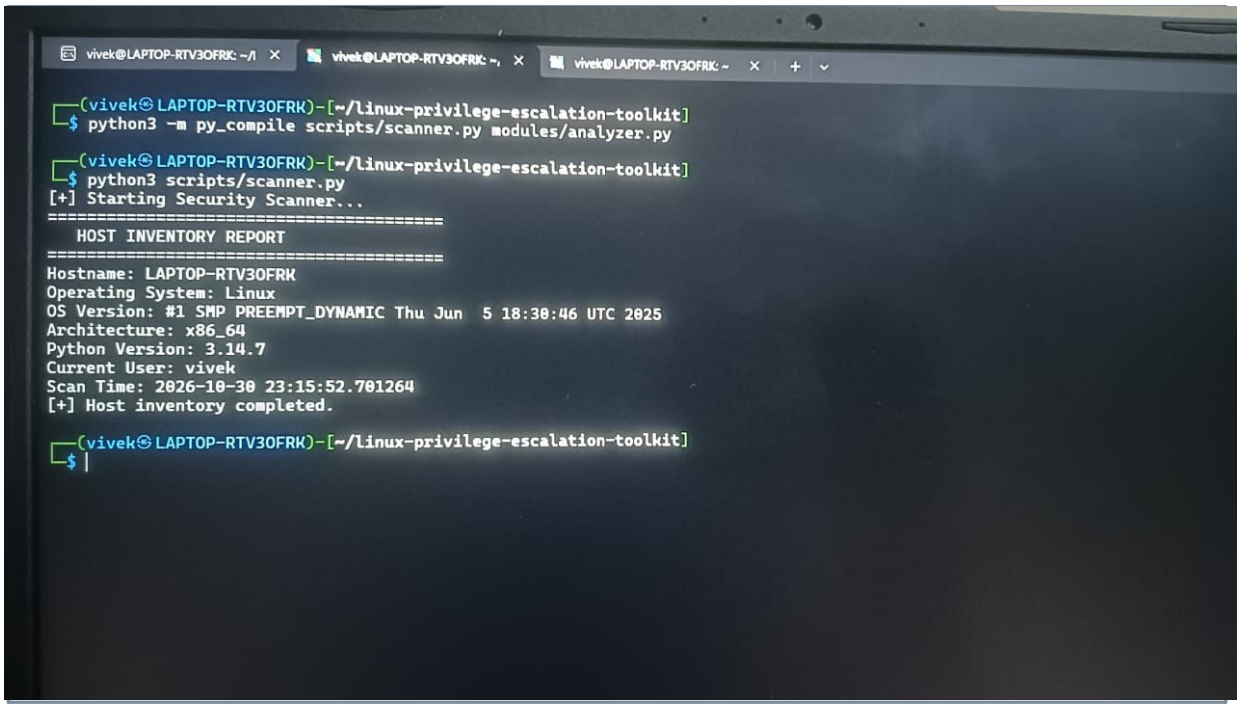




Screenshot 13 — Test Results

Show your completed test table or terminal test output with actual PASS/FAIL results.

Suggested filename: 13_test_results.png



15 | Security, Ethics and Limitations

Authorization

Run the toolkit only on systems you own or are explicitly authorized to assess. A school or employer lab may have specific rules; follow them and keep a record of the permitted scope.

Data handling

• Do not collect or publish passwords, private keys, tokens, or unrelated personal data.

• Redact usernames, hostnames, IP addresses, and sensitive paths where appropriate.

• Do not commit secrets, private logs, or sensitive system output to a public repository.

• Use a private repository if the instructor or organization requires restricted evidence.

Limitations

• A finite set of checks cannot cover every Linux distribution, service, or privilege-escalation condition.

• Permission to read a file may vary; skipped checks are not proof of security.

• SUID/SGID or writable-path indicators may be legitimate and require context.

• The tool does not exploit findings or prove an escalation path.

• Preliminary severity rules may produce false positives and false negatives.

• A clean report does not guarantee that a host is secure.

Defensive value

The project encourages repeatable enumeration, evidence-based reporting, least privilege, patch management, and human verification. These are useful foundations for Linux security assessment and Red Team preparation.

16 | Future Enhancements

• Add unit tests using temporary test directories and synthetic permission metadata.

• Add CSV export and machine-readable JSON schema validation.

• Add configurable checks by distribution, with a documented baseline.

• Add allowlists for known legitimate SUID/SGID files, reviewed by an administrator.

• Add report comparison between two authorized runs to highlight configuration changes.

• Add a summary chart for check status and risk categories.

• Add signed release hashes and versioned releases for reproducibility.

• Add a remediation tracking field: owner, due date, and verification status.

• Add CI tests that run without root privileges.

• Improve documentation for accessibility and beginner-friendly interpretation.

Enhancements should remain defensive and transparent. Automatic exploitation or automatic changes to privileged configuration are intentionally excluded.

17 | Conclusion

The Linux Privilege Escalation Automation Toolkit is a practical introductory project in Linux security auditing. It organizes selected configuration checks into a repeatable workflow, records evidence, and provides preliminary risk and remediation guidance. Its strongest contribution is not proving that a machine can be compromised, but helping a reviewer identify configuration areas that deserve careful validation.

The project should be submitted with source code, a README, actual test results, and the fourteen genuine screenshots specified in this report. The final conclusion must reflect what was actually tested and should state limitations honestly.

18 | References

Use the official documentation below for concepts and APIs. Confirm the current documentation relevant to the Linux distribution and Python version used in your lab.

• Python 3 documentation — https://docs.python.org/3/

• Python pathlib — https://docs.python.org/3/library/pathlib.html

• Python os module — https://docs.python.org/3/library/os.html

• Python json module — https://docs.python.org/3/library/json.html

• Linux man-pages project — https://www.kernel.org/doc/man-pages/

• Ubuntu security documentation — https://ubuntu.com/security

• Debian Security — https://www.debian.org/security/

Add any course handouts, instructor-approved lab references, or additional official sources that you actually used. Do not list references you did not consult.

19 | Final Submission Checklist

• [ ] Cover page has the correct project title and student details.

• [ ] Certificate / mentor section completed only by an authorized person, if required.

• [ ] Source code runs in the authorized lab and matches the report.

• [ ] README explains requirements, run command, output, and safe-use limits.

• [ ] All test cases have actual PASS/FAIL results and short notes.

• [ ] Results section uses observed output, not invented results.

• [ ] All 14 screenshots are inserted, numbered, captioned, and readable.

• [ ] Screenshot captions explain what each image demonstrates.

• [ ] Sensitive host details and secrets are redacted.

• [ ] Report limitations and false-positive risks are clearly stated.

• [ ] GitHub repository contains the correct files and no secrets.

• [ ] PDF opens correctly and page numbers are present.

• [ ] Source code and report are submitted together if required by the institute.

How to strengthen your evaluation

Be ready to explain the architecture, what each check observes, why a flagged configuration needs human review, how errors are handled, what the tool does not do, and one real issue you corrected or learned from. Clear evidence and honest analysis are stronger than exaggerated claims.

Career note

This project can support a fresher cybersecurity portfolio, especially for Linux security and penetration-testing fundamentals. It cannot guarantee a particular mark, interview, job, or salary; outcomes depend on the evaluator, your explanation, other skills, and hiring conditions.

END OF REPORT TEMPLATE